

PROJECT: Market Data Feed Handler

After studying computer microarchitecture, operating systems and networking (by reading Inside the Machine, by Jon Stokes, Operating Systems: Three Easy Pieces, by Andrea and Remzi Arpaci-Dusseau and TCP/IP Illustrated, by Kevin Wall) I have decided to complete a project to test my understanding of all these topics.

This project aims to create an application that parses binary ITCH UDP multicast traffic from a multicast IP that provides market data feeds and stores the data into a kdb table. It aims to use C++ multithreading, packet loss/recovery logic (using sequence number checks to implement a form of reliability of data transfer in UDP, like TCP's sequence numbers), minimized system calls and general context switches to minimize latency and benchmarking tools to measure system behaviour.

I aim to complete this project to enhance my understanding of low latency environments and high frequency trading systems. The project will be completed entirely in C++ and is split up into stages, which will have a diary of entries attached to log all study done, learnings acquired and explanation of methods of implementation used.

Project Plan

1. Create an ITCH server on a different computer on the local network and publish a stream of ITCH messages in UDP packets to a multicast IP.
2. Join the multicast group from step 1, receive UDP multicast traffic and parse the traffic.
3. Use C++ multithreading to separate the multicast receiver (producer) from the packet reader (consumer)
4. Add changes to implement packet loss, out of order detection and recovery strategies, using sequence numbers.
5. Stream the parsed messages to a kdb table

Focus Points

1. Reduce context switches, maximize throughput – we want maximum CPU efficiency, no wasted cycles
2. Maximize concurrency, use multithreading where possible
3. Focus on robustness as much as completeness, handle dropped packets and add comprehensive logging.

Rough Plan

REMEMBER: ROBUSTNESS IS JUST AS IMPORTANT AS COMPLETENESS

Week 1: Foundations, Parsing & Basic Networking (Single Thread)

1. **Understand ITCH/IEX Protocol and Data Formats**
 - Study binary feed formats thoroughly.
 - Collect sample data for offline testing.
 2. **Set up C++ Project & Basic Binary Parsing**
 - Implement parsing logic offline on files to confirm correctness.
 - Unit test parsing thoroughly.
 3. **Create UDP Multicast Receiver (Single Threaded)**
 - Setup multicast socket with appropriate options (`SO_RCVBUF`, `SO_REUSEADDR`).
 - Implement simple `recvfrom` or `recvmsg` loop receiving packets.
 4. **Parse Incoming Packets Inline**
 - Parse packets immediately after reception, still in the main thread.
 - Print debug info to validate the integration of parsing and reception.
-

Week 2: Introduce Multithreading & Decouple I/O from Parsing

5. **Separate Networking from Parsing Using Producer-Consumer Model**
 - Create a **network thread** dedicated to reading UDP packets and pushing raw data into a **thread-safe queue** (e.g., **lock-free** or **ring buffer**).
 - Create one or more **parsing threads** to consume packets from this queue and parse them concurrently.
 6. **CPU Affinity & Thread Pinning**
 - Pin network thread and parsing threads to different CPU cores to avoid contention.
 - Use `pthread_setaffinity_np` or `std::thread::native_handle` + OS-specific APIs.
 - Measure performance impact.
 7. **Batch Processing & Bulk Reads**
 - Use `recvmsg` in the network thread to receive multiple packets per syscall to reduce overhead.
 - Push batches into the queue or distribute to parsing threads.
-

Week 3: Sequence Number Checking & Packet Loss Logic in Concurrent Setup

8. **Implement Sequence Number Tracking in Parsing Threads**
 - Track sequence numbers independently or coordinate through shared state (use atomic operations or thread-safe structures).
 - Handle out-of-order packets and loss detection.
9. **Packet Loss Handling & Recovery Strategies**
 - Implement logic for reporting or handling missing packets.
 - Consider asynchronous recovery requests if feed supports.
10. **Load Balancing Parsing Threads**
 - Tune number of parsing threads based on CPU cores and workload.

- Consider work-stealing queues or other scalable concurrent data structures.
-

Week 4: High-Performance Optimizations & System Tuning

11. Reduce Context Switches and Maximize Throughput

- Experiment with busy polling (`SO_BUSY_POLL`), spinlocks, or hybrid sleep/wake schemes in network thread.
- Use huge pages or lock memory if needed for shared buffers.

12. Optimize Buffering

- Implement or use a highly efficient circular buffer for passing messages from parsing threads to downstream consumers.
- Ensure zero-copy or minimal-copy transfers where possible.

13. Measure & Tune Socket Buffer Sizes & Thread Priorities

- Adjust `SO_RCVBUF`, CPU priorities, and scheduling policies (`SCHED_FIFO` or `SCHED_RR`) for critical threads.
-

Week 5: Data Output, Integration & Final Polishing

14. Stream Parsed Messages to kdb+ or Circular Buffer

- Implement a dedicated **output thread(s)** consuming parsed data and streaming into kdb+ via IPC or other efficient mechanisms.
- Buffer output to minimize blocking and syscalls.

15. Full System Benchmark & Profiling

- Profile using `perf`, `htop`, and `strace` to identify bottlenecks.
- Verify CPU utilization close to 100% across all cores dedicated to the app.
- Validate latency and packet loss under high throughput.

16. Robustness & Error Handling

- Handle dropped packets, reconnections, and any edge cases gracefully.
- Add comprehensive logging and metrics for monitoring.

Week 1

- Understand ITCH/IEX protocol and data formats
- Setup the C++ project and create a UDP Multicast receiver using Berkeley sockets API for C++
- Parse the incoming packets in the main thread
- Once tested, create a separate network thread for reading UDP packets and pushing the data into a thread-safe queue
 - Use 1+ parsing threads to consume the packets concurrently, using synchronization primitives (locks, semaphores, CV's etc)
- Use CPU affinity to pin the threads to different cores to avoid resource contention with unrelated processing (reading from buffer vs reading to buffer)

- Enable batch processing for the NIC, so there are less context switches to the kernel for reading into the buffer

Week 2

- Track the sequence numbers of each UDP packet and handle out of order packets/loss detection
 - o Implement logic for handling missing packets
- Implement tuning for number of parsing threads based on CPU cores and workload
- Tweak network thread to reduce context switches where possible
 - o Increased page size if supported for shared buffer
- Stream parsed messages to a kdb table, via IPC (buffer output to minimize blocking/system calls)

Week 3

- Use perf, htop and strace to identify bottlenecks and verify CPU utilization close to 100% across all cores dedicated for running parsing and network threads
- Validate latency and packet loss under high throughput